Internship – 2026

By

Uptoskills

Duration :- 3 months

Start Date:- 24-09-2026

End Date:- 24-12-2026

Intern Code:- US/HR/2026/6WZML6

Name:- Aharnish Dhaw Kumar

Course:- Cyber Security Intern

Enroll No :- AUJ254692

Interview date: 22-09-2026

Offer letter date: 23-09-2026

Leader Assigned on: 26-09-2026

Project Handler: Rudrapratap Singh Bais

Asst Project Handler: Shamim Aftab Ansari

Project Name: Cyber Security

Project Briefed: 26-09-2026

Task 5 OWASP ZAP (Zet Attack Proxy)

Date of allotment: 13/10/2026

Date of completion: 16/10/2026

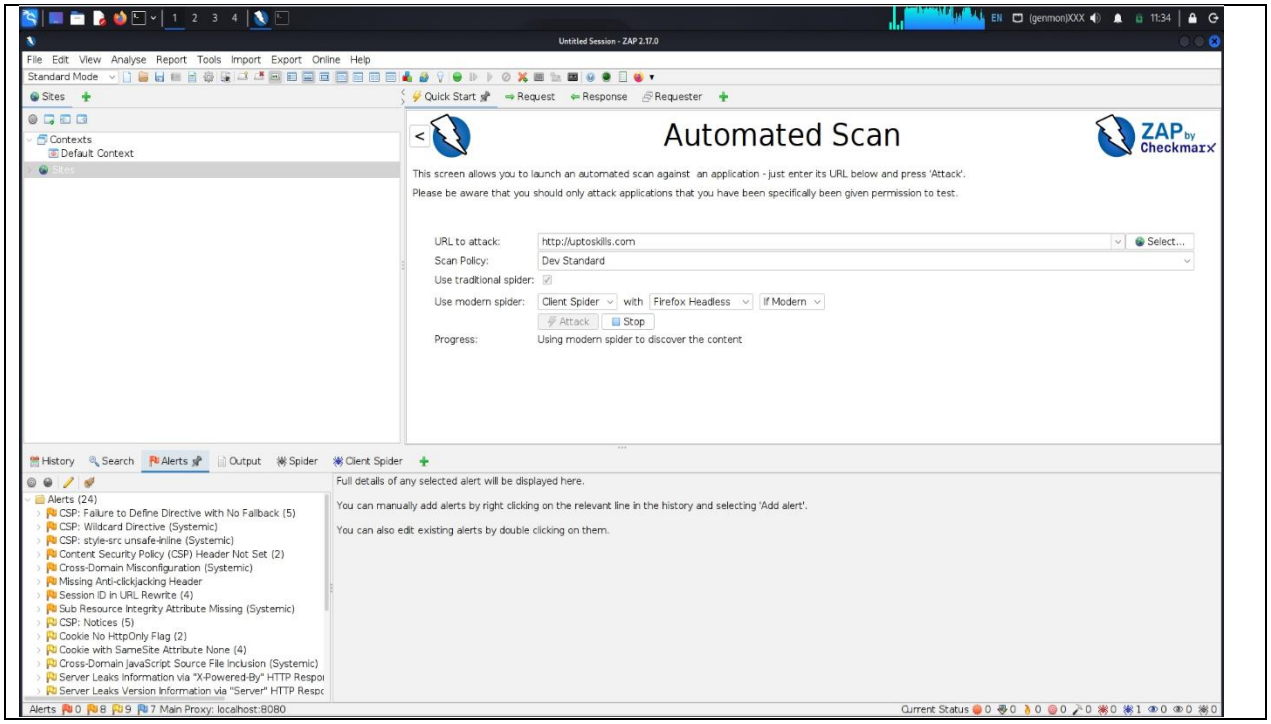
Deadline: 18/10/2026 (5days)

1) **What is OWASP ZAP (Zet Attack Proxy)?**
   - OWASP ZAP (Zed Attack Proxy) is a free, open-source web application security testing tool used to find security vulnerabilities in websites and web applications.
   - It is widely used by cybersecurity analysts, ethical hackers, and penetration testers to perform Vulnerability Assessment and Penetration Testing (VAPT).
   - Vulnerability scanning: Identifies potential security weaknesses in web applications.
   - Intercepting HTTP/HTTPS traffic: Lets you inspect requests and responses between your browser and a web application.
   - Passive scanning: Analyzes traffic for potential issues without deliberately attacking the target.
   - Active scanning: Sends specially crafted requests to test for certain vulnerabilities. Only use this on systems you own or have explicit permission to test.
   - Security reporting: Generates reports containing findings, risk levels, and supporting evidence.

2. Examples of vulnerabilities ZAP can help identify

Vulnerability or issue	What it means
Cross-Site Scripting (XSS)	Untrusted input may execute JavaScript in a browser.
SQL injection	User input may interfere with database queries.
Missing security headers	Important browser security protections may be absent.
Cookie security issues	Cookies may lack protections such as `Secure` or `HttpOnly`.
Information disclosure	Responses may expose sensitive technical details.
Server configuration issues	The application may have insecure settings.

1) **Screenshot of zap ui scan**



Vulnerability Assessment Report

Assessment Details

- **Tool Used:** OWASP ZAP 2.17.0
- **Target URL:** http://uptoskills.com
- **Scan Type:** Automated Scan
- **Scan Policy:** Dev Standard
- **Spider Used:** Traditional Spider + Modern Client Spider (Firefox Headless)
- **Assessment Methodology:** Automated Web Application Security Testing

---

Executive Summary

A vulnerability assessment was performed on the target web application using OWASP ZAP. The scan identified multiple security misconfigurations and missing security headers that could expose the application to attacks such as clickjacking, session hijacking, information disclosure, and content injection.

Several findings were categorized as medium-to-low risk security weaknesses. While no critical exploits are visible in the screenshot, remediation is recommended to improve the application's security posture.

Findings

1. Content Security Policy (CSP) Header Not Set

Description

The application does not define a Content Security Policy header.

Risk

Medium

Impact

- Increased exposure to Cross-Site Scripting (XSS).
- Malicious scripts may execute within the browser.

Recommendation

Implement a restrictive CSP header such as:

HTTP

Content-Security-Policy:

default-src 'self';

script-src 'self';

style-src 'self';

object-src 'none';

---

2. CSP: Wildcard Directive

Description

The CSP configuration contains wildcard (*) directives.

Risk

Low-Medium

Impact

- Allows content loading from untrusted domains.
- Weakens protection offered by CSP.

Recommendation

Replace wildcard domains with specific trusted sources.

---

3. CSP: style-src unsafe-inline

Description

The CSP permits inline CSS using 'unsafe-inline'.

Risk

Low

Impact

- Inline styles can potentially be abused during client-side attacks.

Recommendation

Use external CSS files and remove:

HTTP

style-src 'unsafe-inline'

---

4. Missing Anti-Clickjacking Header

Description

The website lacks clickjacking protection.

Risk

Medium

Impact

Attackers may embed the application within malicious iframes.

Recommendation

Configure:

HTTP

X-Frame-Options: DENY

or

HTTP

Content-Security-Policy: frame-ancestors 'none';

---

5. Session ID in URL Rewrite

Description

Session identifiers are exposed through URL parameters.

Risk

Medium

Impact

- Session Hijacking
- Session Fixation
- Leakage through browser history and logs

Recommendation

Store session identifiers in secure cookies:

HTTP

Set-Cookie:

HttpOnly;

Secure;

SameSite=Strict

---

6. Cookie Without HttpOnly Flag

Description

Cookies are missing the HttpOnly attribute.

Risk

Medium

Impact

Client-side scripts can access session cookies if an XSS vulnerability exists.

Recommendation

Enable:

HTTP

Set-Cookie: HttpOnly

---

7. Cookie with SameSite Attribute None

Description

Cookies are configured with SameSite=None.

Risk

Low-Medium

Impact

May increase exposure to Cross-Site Request Forgery (CSRF).

Recommendation

Use:

HTTP

SameSite=Lax

or

HTTP

SameSite=Strict

where appropriate.

---

8. Subresource Integrity Attribute Missing

Description

External scripts/styles do not use Subresource Integrity (SRI).

Risk

Low

Impact

Compromised third-party resources may execute malicious code.

Recommendation

Implement integrity checks:

HTML

example.jsscript>

---

9. Cross-Domain JavaScript Source File Inclusion

Description

JavaScript files are loaded from external domains.

Risk

Low

Impact

Trust is placed on third-party infrastructure.

Recommendation

- Use trusted CDN providers.
- Apply Subresource Integrity (SRI).

---

10. Information Disclosure via HTTP Headers

Description

Server information is disclosed through:

- Server Header
- X-Powered-By Header

Risk

Low

Impact

Attackers can identify technologies and versions used.

Recommendation

Remove or mask headers:

HTTP

Server:

X-Powered-By:

---

11. Cross-Domain Misconfiguration

Description

Improper cross-domain configuration detected.

Risk

Medium

Impact

Could enable unauthorized resource access from external domains.

Recommendation

Review CORS configuration:

HTTP

Access-Control-Allow-Origin:

Allow only trusted domains.

Risk Summary

Severity	Count

High	0
Medium	5
Low	6
Informational	Multiple

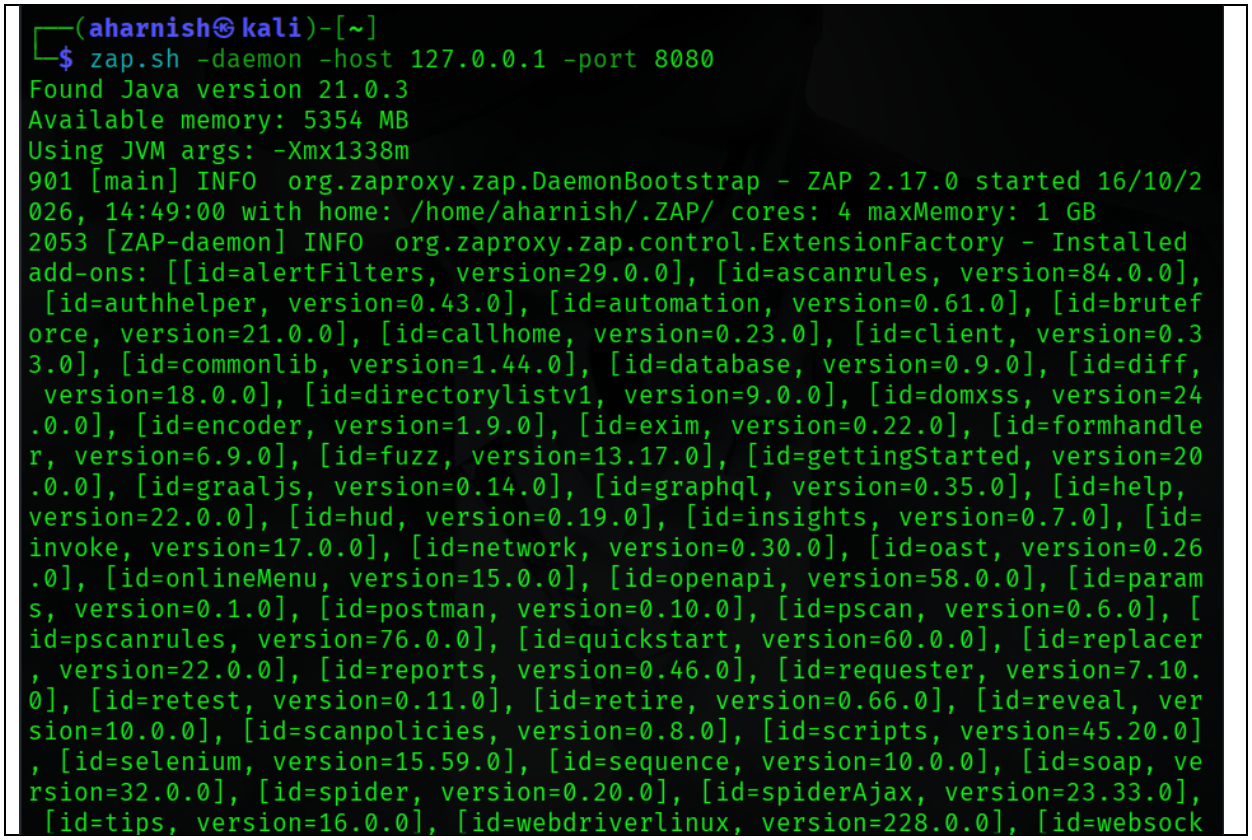
Conclusion

The OWASP ZAP assessment identified several security header misconfigurations, cookie security issues, session management weaknesses, and information disclosure vulnerabilities. No critical vulnerabilities are visible from the screenshot; however, the discovered weaknesses could assist attackers during reconnaissance or facilitate further exploitation.

The highest priority remediation activities should focus on:

1. Implementing a strong Content Security Policy.
2. Enabling HttpOnly and Secure cookie flags.
3. Eliminating session IDs from URLs.
4. Adding clickjacking protection.
5. Disabling server version disclosure.

2) Screenshot of Running OWASP ZAP in Daemon Mode



Practical Report: Running OWASP ZAP in Daemon Mode

Experiment Title

Running OWASP ZAP in Daemon Mode Using Command Line Interface (CLI)

Aim

To start OWASP ZAP in daemon mode on Kali Linux and configure it to listen on a local host and specified port for automated security testing.

Objective

- Launch OWASP ZAP without the graphical interface.
- Configure ZAP to run as a background service.
- Enable API-based and automated vulnerability scanning.
- Verify successful startup of the ZAP daemon.

Requirements

- Kali Linux
- Java Runtime Environment (JRE)
- OWASP ZAP Installed
- Terminal Access
- Sudo/User Privileges

Command Used

Shell

```
zap.sh -daemon -host 127.0.0.1 -port 8080
```

Command Explanation

Parameter	Description
zap.sh	Launches OWASP ZAP
-daemon	Starts ZAP without GUI in background mode
-host 127.0.0.1	Binds the service to localhost
-port 8080	Sets the listening port to 8080

Procedure

Step 1

Open the Kali Linux terminal.

Step 2

Execute the following command:

Shell

```
zap.sh -daemon -host 127.0.0.1 -port 8080
```

Step 3

Wait for ZAP initialization.

Step 4

Observe startup messages including:

- Java version detection
- Available memory allocation
- JVM settings
- ZAP version details
- Loaded add-ons/modules

Step 5

Confirm successful startup by checking the message indicating that the daemon is running and listening on the configured host and port.

Screenshot Analysis

Figure 2: Starting OWASP ZAP in Daemon Mode

The screenshot shows:

- Command execution:

  Shell

  zap.sh -daemon -host 127.0.0.1 -port 8080
- Java Version Detected:

  Plain Text

  Java version 21.0.3
- Available System Memory:

  Plain Text

  5354 MB
- JVM Memory Allocation:

  Plain Text

  -Xmx1338m
- ZAP Version Started:

  Plain Text

  OWASP ZAP 2.17.0
- Host Configuration:

  Plain Text

  127.0.0.1

- Port Configuration:

  Plain Text

  8080
- Numerous add-ons successfully loaded, including:
  - alertFilters
  - ascanrules
  - automation
  - bruteforce
  - database
  - fuzz
  - openapi
  - postman
  - reports
  - spider
  - spiderAjax
  - selenium
  - scripts
  - websocket

Observation

1. OWASP ZAP started successfully in daemon mode.
2. Java Runtime Environment was detected correctly.
3. Memory allocation was configured automatically.
4. ZAP loaded multiple security testing extensions and add-ons.

5. The application initialized on localhost (127.0.0.1) and port 8080.
6. No startup errors were observed during initialization.

Result

OWASP ZAP was successfully launched in daemon mode using the command:

Shell

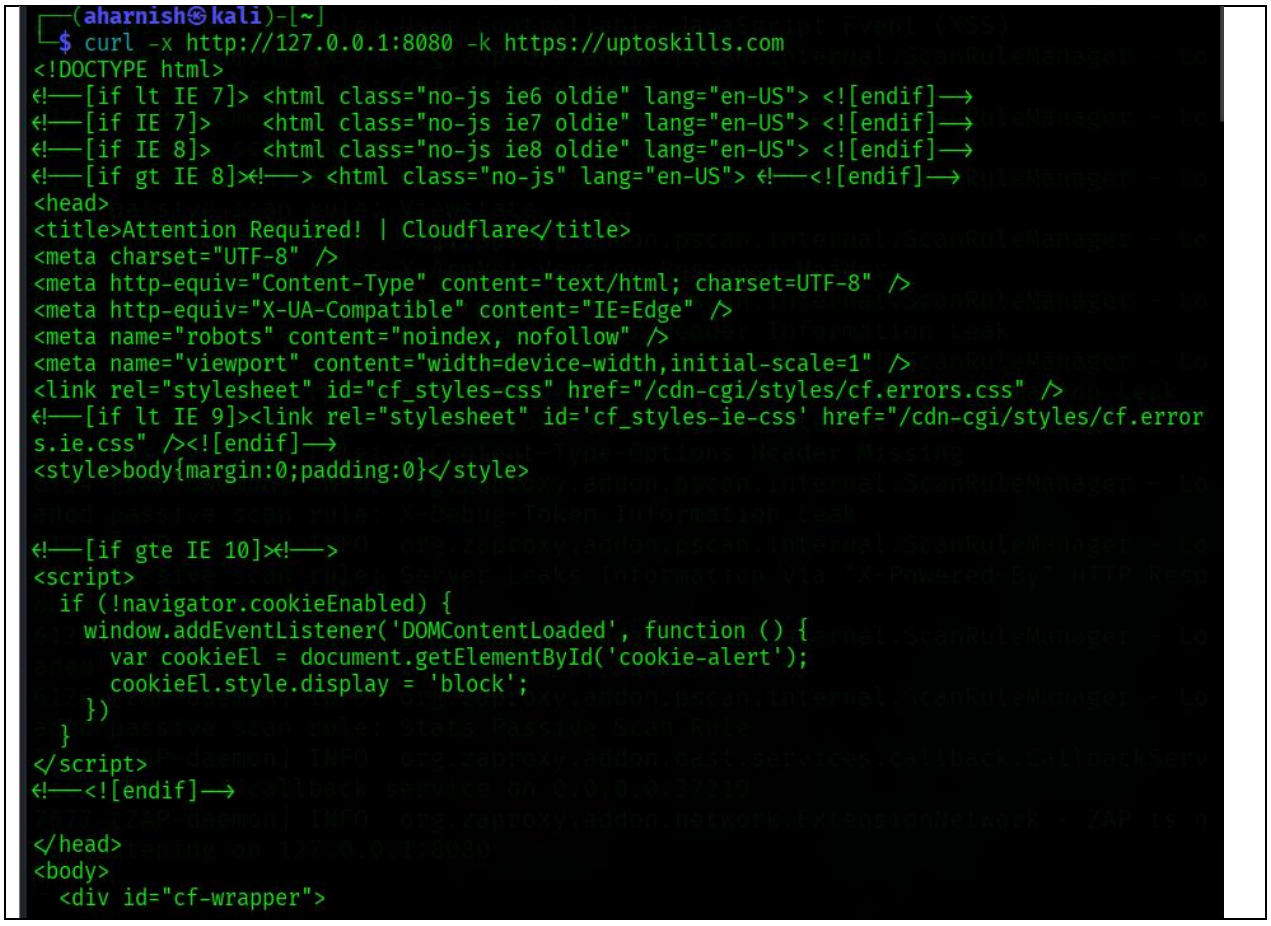
zap.sh -daemon -host 127.0.0.1 -port 8080

The daemon initialized properly, detected Java 21.0.3, allocated memory resources, and loaded all required security testing modules. The system is now ready for automated vulnerability scanning, API integration, and web application security assessments.

Conclusion

The experiment successfully demonstrated how to start OWASP ZAP in daemon mode on Kali Linux. The tool initialized correctly, loaded essential security modules, and became available on localhost port 8080 for automated penetration testing and vulnerability assessment activities.

3) Screenshot of curl -x http://127.0.0.1:8080 -k https://uptoskills.com



Practical Report: Testing Web Server Response Through OWASP ZAP Proxy

Experiment Title

Accessing a Target Website Through OWASP ZAP Proxy Using cURL

Aim

To verify that web traffic can be routed through the OWASP ZAP proxy and to observe the HTTP response received from a target website.

Objective

- Connect to a target website using the cURL tool.
- Route traffic through the OWASP ZAP proxy running on localhost.
- Examine the HTTP response returned by the web server.
- Validate that the proxy is intercepting and forwarding requests.

Requirements

- Kali Linux
- OWASP ZAP Running in Daemon Mode
- cURL Utility
- Internet Connectivity

Command Used

Shell

curl -x http://127.0.0.1:8080 -k https://uptoskills.com

``

Command Explanation

Parameter	Description
curl	Command-line tool for transferring data
-x http://127.0.0.1:8080	Routes traffic through the OWASP ZAP proxy
-k	Ignores SSL certificate verification
https://uptoskills.com	Target website being accessed

Procedure

Step 1

Ensure OWASP ZAP is running in daemon mode on port 8080.

Step 2

Open a terminal window.

Step 3

Execute the command:

Shell

curl -x http://127.0.0.1:8080 -k https://uptoskills.com

Step 4

Observe the HTML content returned by the server.

Step 5

Verify that the request passes through the ZAP proxy and reaches the target website.

Screenshot Analysis

Figure 3: Accessing Target Website Through ZAP Proxy

The screenshot shows:

- Successful execution of:

Shell

curl -x http://127.0.0.1:8080 -k https://uptoskills.com

- The website returns an HTML response beginning with:

HTML

<!DOCTYPE html>

<html lang="en-US">

- Web page title displayed:

HTML

<title>Attention Required! | Cloudflare</title>

- Various HTML meta tags and stylesheet references were received from the server.
- The output confirms that an HTTP response was successfully delivered through the configured proxy.

Observation

1. The cURL request was successfully sent through the OWASP ZAP proxy at **127.0.0.1:8080**.
2. The target website responded with HTML content.
3. The response page displayed a **Cloudflare "Attention Required!"** page.
4. Cloudflare protection likely identified the request as automated traffic and presented a security challenge.
5. This confirms that the proxy is operational and capable of forwarding web requests.

Result

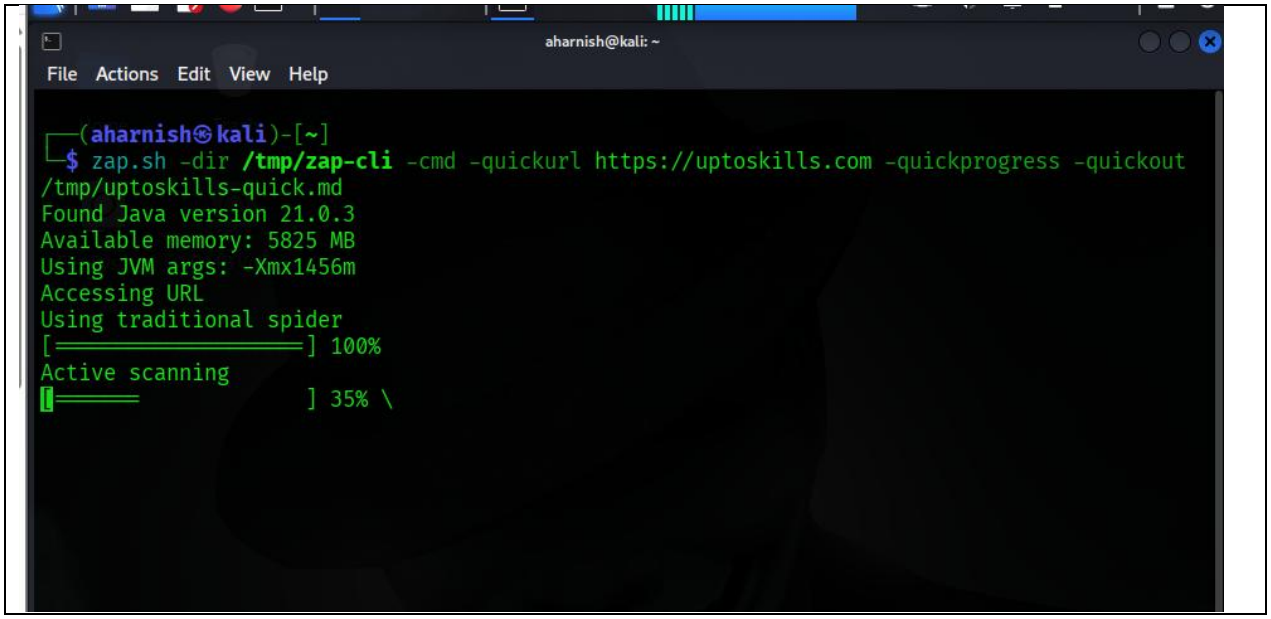
The website request was successfully forwarded through the OWASP ZAP proxy using cURL. The server returned an HTML response containing a Cloudflare protection page, demonstrating that:

- The proxy configuration is functioning correctly.
- Traffic is successfully passing through OWASP ZAP.
- The target server is reachable through the proxy.

Conclusion

- The experiment successfully demonstrated routing web traffic through OWASP ZAP using the cURL utility. The returned Cloudflare page confirmed that the request reached the target server via the proxy. This validates the proper operation of the OWASP ZAP daemon and establishes a foundation for web application security testing and traffic analysis.

4) Screenshot of Run a Quick **Scan with Progress Bars**



Practical Report: Quick Scan Using OWASP ZAP CLI

Aim

To perform a quick security scan on a target website using OWASP ZAP Command Line Interface.

Command Used

Shell

zap.sh -dir /tmp/zap-cli -cmd -quickurl https://uptoskills.com -quickprogress -quickout /tmp/uptoskills-quick.md

Procedure

1. Open the terminal.
2. Run the above command.
3. Allow ZAP to crawl the target website.
4. Monitor the scan progress displayed on the screen.
5. Save the scan report to the specified output file.

Observation

- OWASP ZAP detected Java Version 21.0.3.
- The target URL was accessed successfully.
- Traditional spidering completed 100%.
- Active vulnerability scanning started and reached 35% at the time of the screenshot.
- Scan results were being saved to the specified report file.

Result

OWASP ZAP successfully initiated a quick scan of https://uptoskills.com, completed spidering, and began active security testing.

Conclusion